Subject outline

Lecture topics (by week)

1 – Introduction to quantum computing and maths basics
2 – Single qubit representations and logic operations
3 – Two qubit states and logic gates
4 – Multi-qubit states and quantum arithmetic
5 – Basic quantum algorithms
6 – Period finding, cryptography and quantum factoring
7 – Shor's algorithm, post-quantum crypto, quantum key distribution
8 – Quantum search algorithms
9 – Grover search applications, optimisation problems
10 – Solving optimisation problems on quantum computers
11 – Applications in quantum machine learning
12 – Real quantum computer devices

Assignment schedule:
#1: Hand out in Week 2
#2: Hand out in Week 8

Lecture 13
Shor's factoring algorithm

Recap of the last two lectures

Last week recap: Factoring and period finding

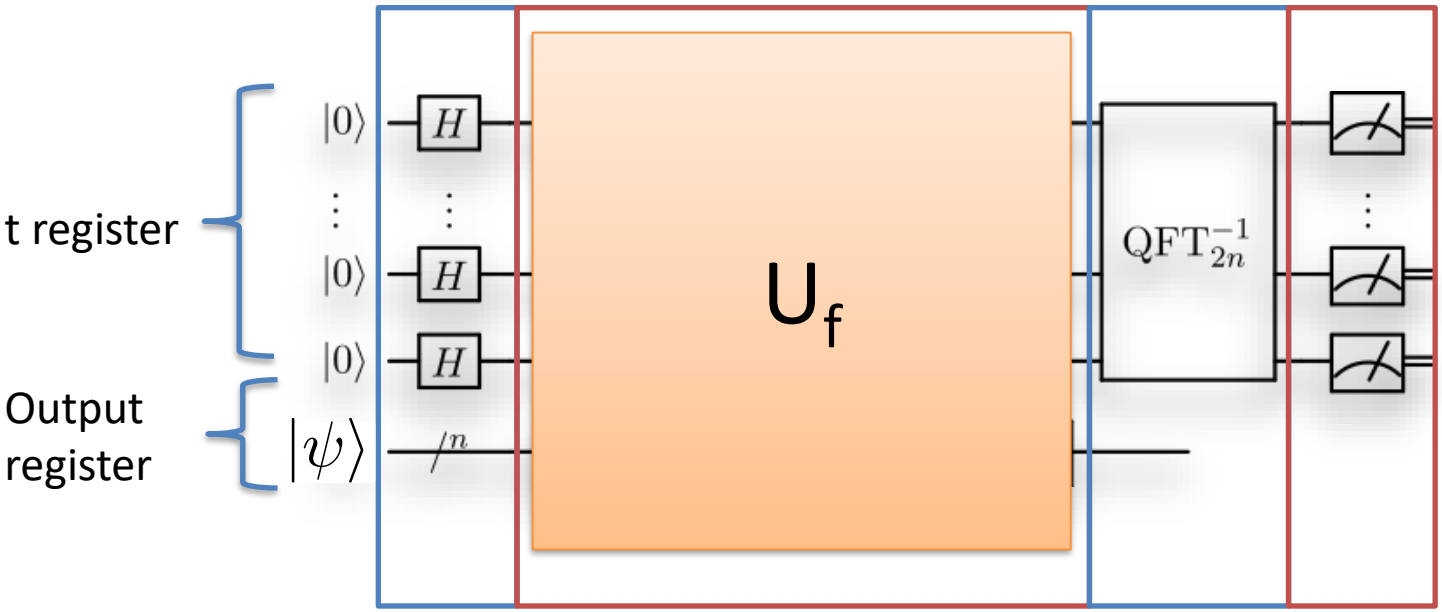
If we can find the period of

$$f(k) = a^k \bmod N$$

efficiently, we can factor N efficiently.

If we can find this period efficiently, and we can then use classical techniques to factor semi-prime numbers into their prime factors.

Quantum Phase Estimation as a function



$$U\,|t\rangle\,|\psi\rangle = |t\rangle\,\exp(i2\pi\theta t)\,|\psi\rangle$$

Quantum computers find periods of periodic functions efficiently!

Period of which Function?

So what if we use a quantum computer to find the period of modular exponentiation,

$$f(k) = a^k \mod N$$

This is Shor's algorithm!

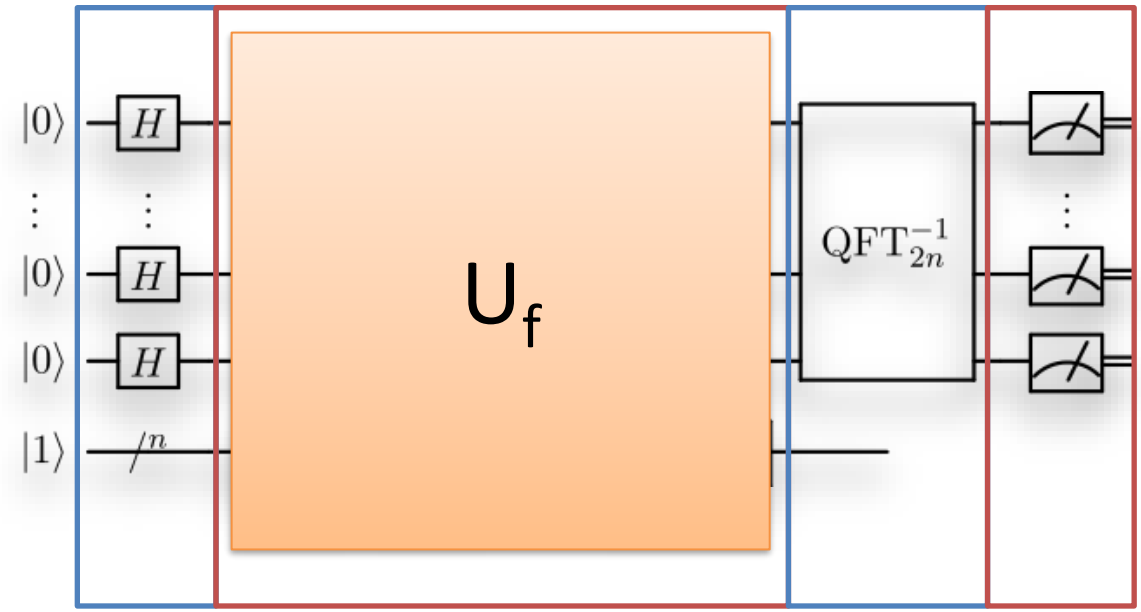
13.1 Shor's algorithm

Shor's algorithm

Two registers*:

2n qubits

n=L qubits



This is an application of the quantum phase estimation circuit which you saw last lecture.

(1) Equal superposition

(2) Calculate function:
$f(x) = a^x \bmod N$

(3) QFT

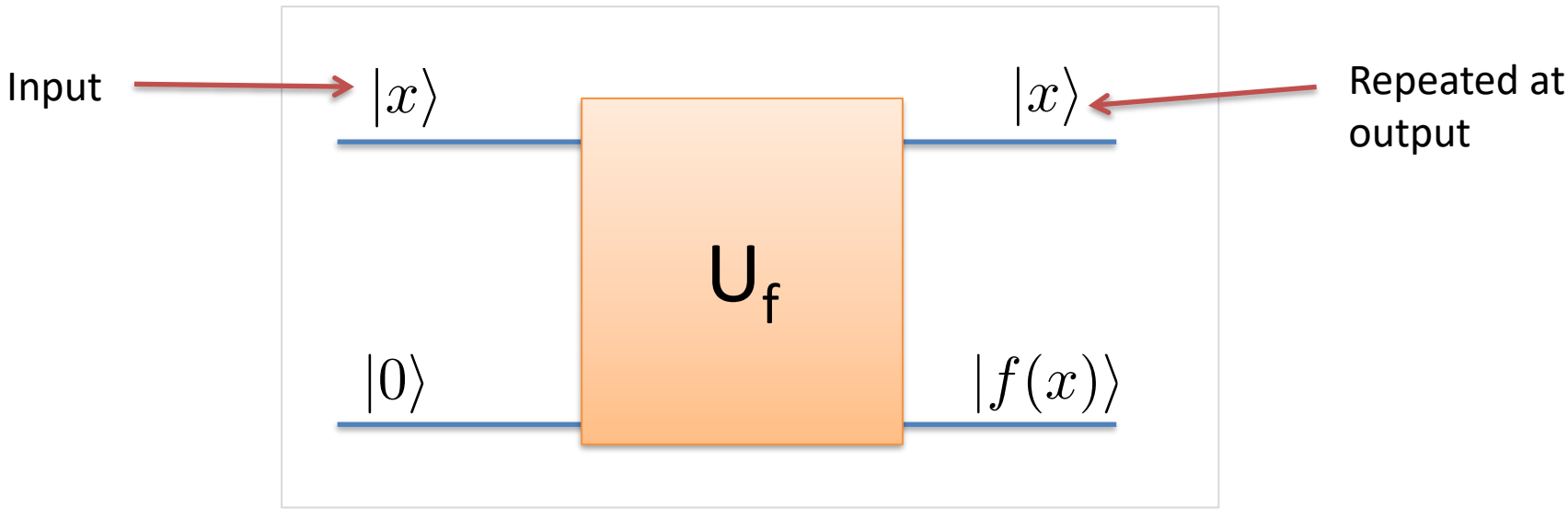
(4) Measure result

* L = number of bits in N

Shor, Proc 35th Ann Symp of Comp Sci, 26, (1995)

Recap: Implementing irreversible functions

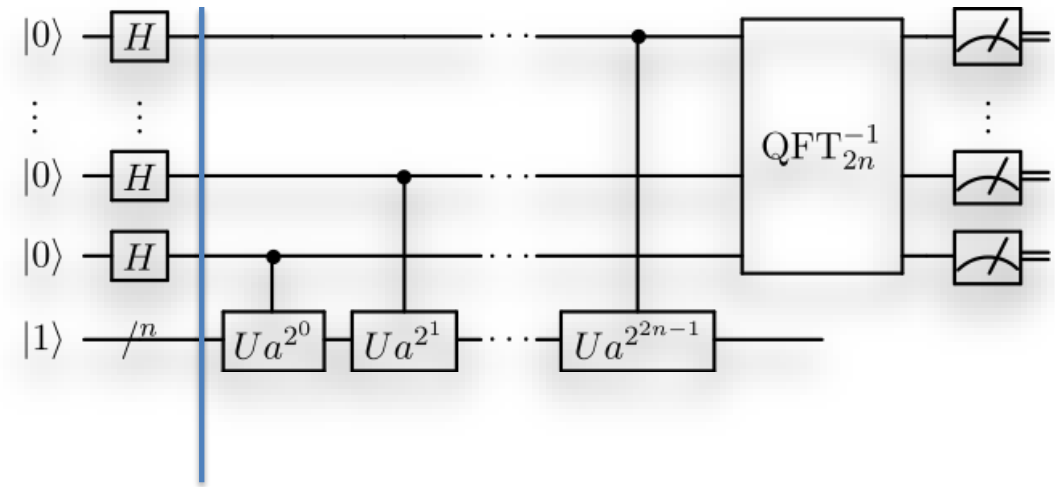
Step 2 is to calculate the modular exponentiation function. As we have seen in previous lectures, we can implement functions:



(2) Calculate function: **f(x) = a^x mod N**

The following slides show how to efficiently perform this function. It might look a bit complicated, but it is simply evaluating this modular exponentiation function.

Shor's algorithm explained



After the Hadamard gates, the top register is in the equal superposition:

$$|\psi\rangle = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{N-1} |x\rangle |1\rangle$$

Recap: Factoring and period finding

Assume we want to factor N=15. Take a number a=2 (say) relatively-prime to N (ie. no prime factors in common) and find *the order* r of a. That is the least r>0, such that $a^r = 1 \bmod 15$:

$$2^0 = 1 \mod 15$$
$$2^1 = 2 \mod 15$$
$$2^2 = 4 \mod 15$$
$$2^3 = 8 \mod 15$$
$$2^4 = 1 \mod 15$$

…after which this pattern repeats.

Formally, we say: the **order** of 2 mod 15 is 4. Or, if we defined the function

$$f(k) = a^k \mod N$$

…we would say that the **period** of f is r, since *f(k+r) = f(k).*

Recap: Example of finding factors from a period

In our case, we have *a=2*, *N=15* and *r=4*. Happily *r=4* is even. So we can rearrange and factorise as follows

$$a^r = 1 \mod N$$
$$a^r - 1 = 0 \mod N$$
$$(a^{r/2} + 1)(a^{r/2} - 1) = 0 \mod N$$

In our case,

$$a^{r/2} - 1 = 2^{4/2} - 1 = 3$$
$$a^{r/2} + 1 = 2^{4/2} + 1 = 5$$

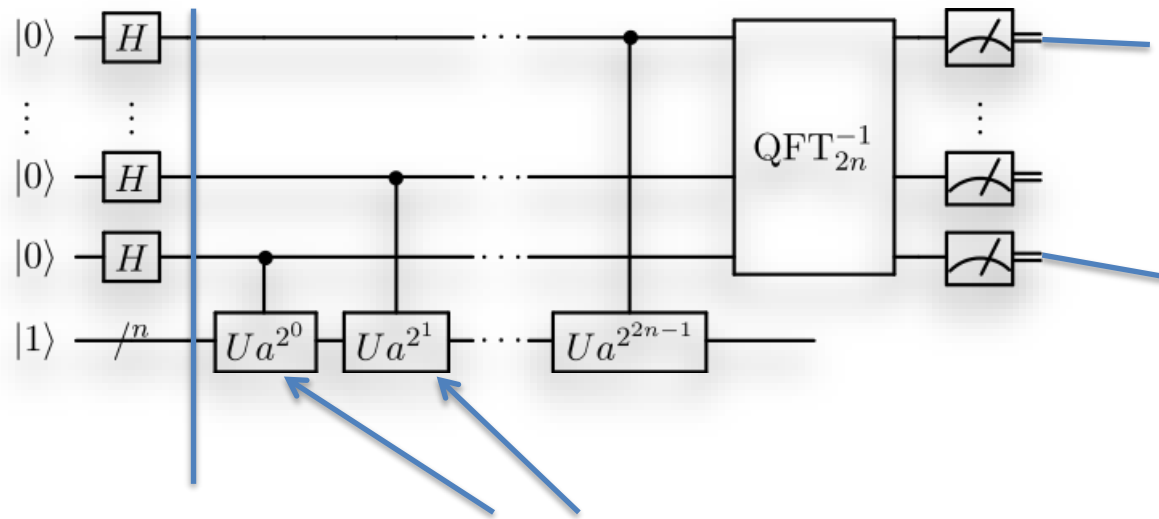
and

$$\boxed{3 \times 5 = 15}$$

Modular exponentiation

2L qubits

L qubits



Most Significant Bit

Least Significant Bit

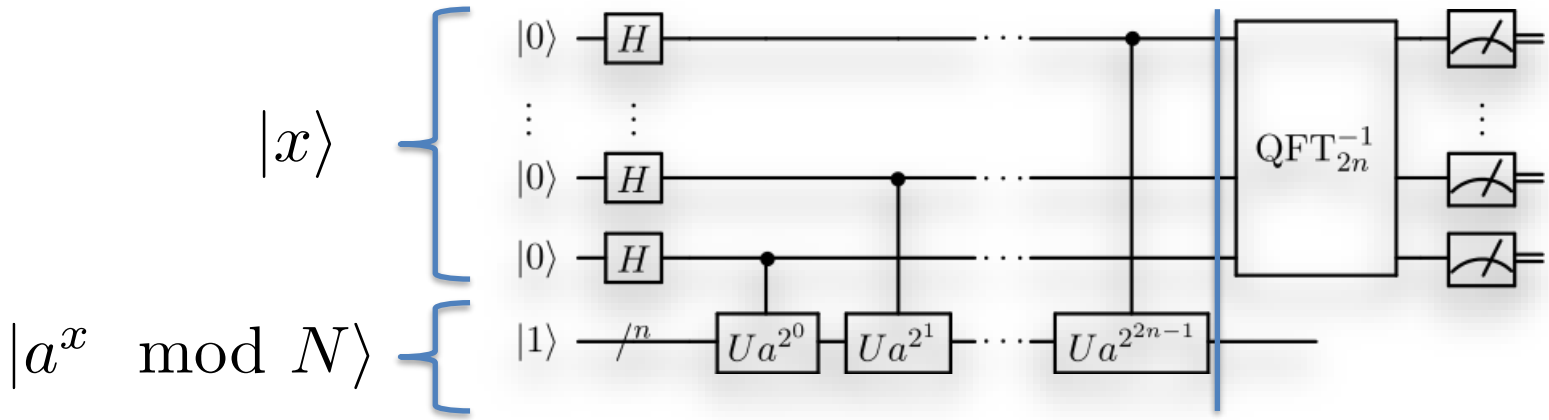
Unitary gates perform multiplication by a^1, a^2, a^4 … *mod N*

For example if the top register contained $x = 101_2$, and $a=2$, and $N=15$ then we would:

- Start with bit *1:*
- Multiplying by $a^1=2^1=2$ gives *2* mod *15*
- bit 0: Do *not* multiply by $a^2=2^2=4$
- bit 1: Multiplying by $a^4=2^4=16$ gives *32,* or *2* mod *15*

Top register in superposition, so bottom register is correlated (entangled) with the top register

Example of modular exponentiation



After modular exponentiation:

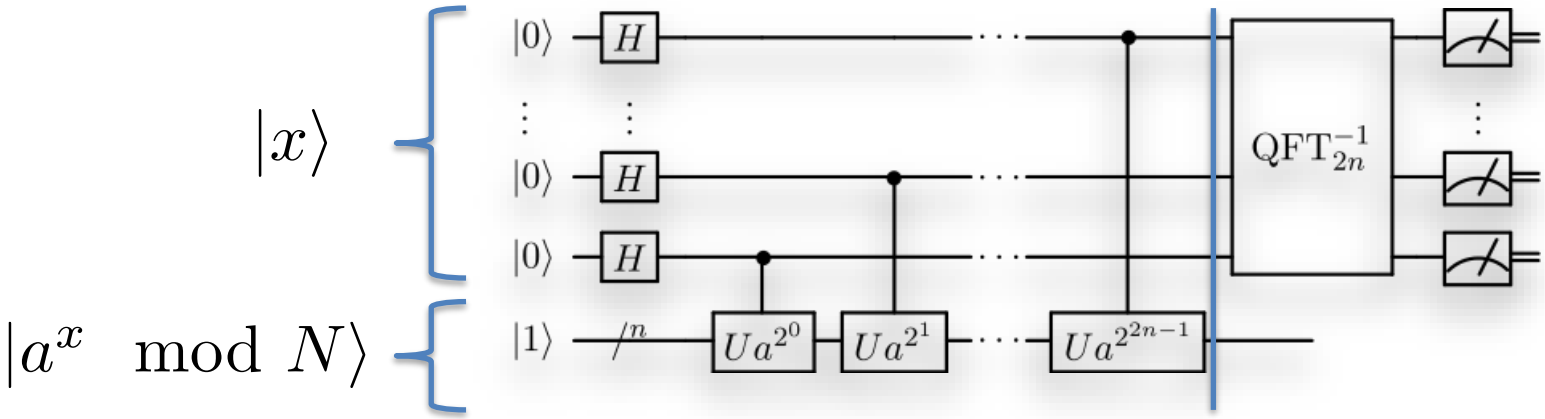
$$|\psi\rangle = \sum_x |x\rangle|a^x \mod N\rangle$$

Note: States are unnormalized (for simplicity)

e.g. For *a=2, N=15*:

$$\begin{aligned}|\psi\rangle = &(|0\rangle + |4\rangle + |8\rangle + |12\rangle) \otimes |1\rangle \\ &+ (|1\rangle + |5\rangle + |9\rangle + |13\rangle) \otimes |2\rangle \\ &+ (|2\rangle + |6\rangle + |10\rangle + |14\rangle) \otimes |4\rangle \\ &+ (|3\rangle + |7\rangle + |11\rangle + |15\rangle) \otimes |8\rangle\end{aligned}$$

Shor's algorithm explained



If, at this point the bottom register is *measured* to be *2* (at random), we may collapse to the state to the following

$$|\psi\rangle = (|1\rangle + |5\rangle + |9\rangle + |13\rangle) \otimes |2\rangle$$

Recap: Divisors of N

In our case 3 and 5 divide N=15 exactly, but we're not guaranteed that always, only that:

$$(a^{r/2}+1)(a^{r/2}-1) = 0 \mod N$$

ie. that

$$(a^{r/2}+1)(a^{r/2}-1) = kN$$

As long as neither factor is a multiple of N, then both will have non-trivial factors with N. To find these factors, we find the greatest common divisors (for which the Euclidean algorithm is efficient):

$$gcd(a^{r/2}+1, N)$$

$$gcd(a^{r/2}-1, N)$$

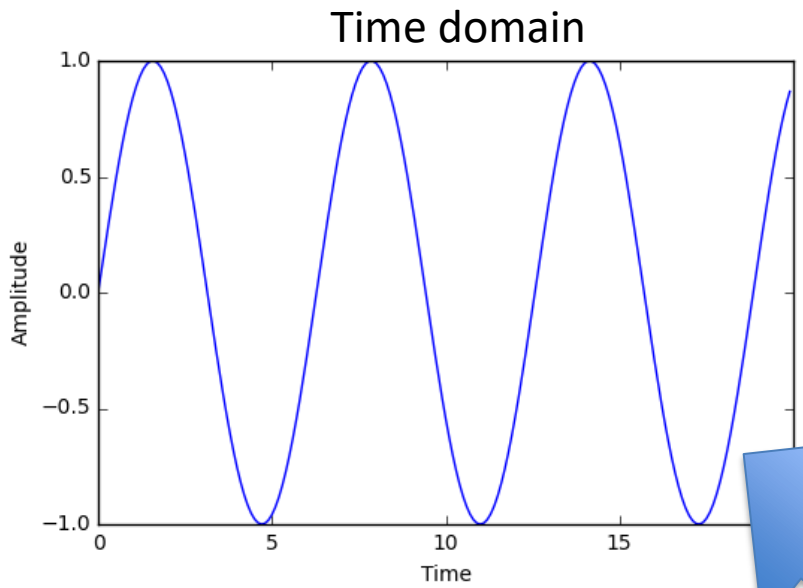
These give a **non-trivial factor of N**.

If r is odd or if the factors found are trivial, we repeat the algorithm with a different choice of a.

The quantum Fourier transform

13.2 QFT for Shor's algorithm

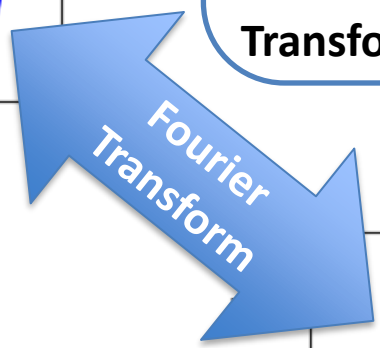
Introduction to Fourier transform

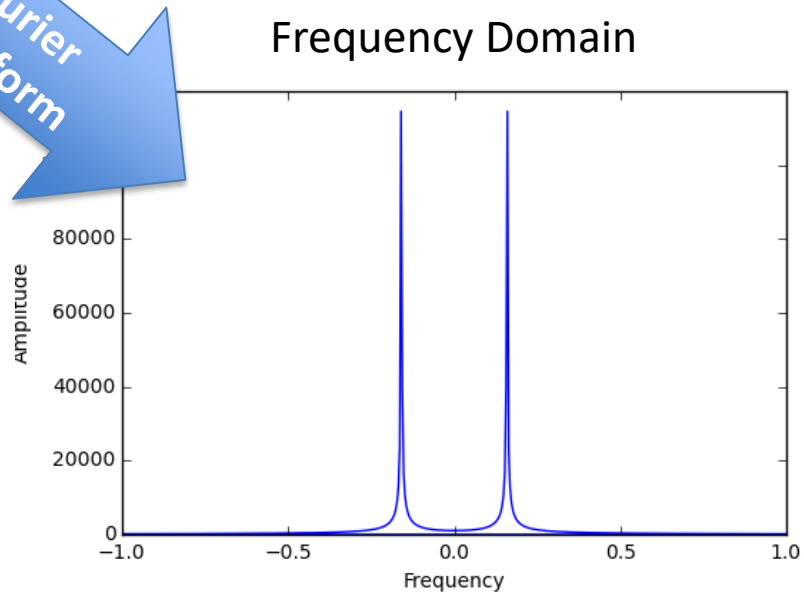


If we have a **periodic function**, and want to know:

What is the period/frequency of this function?

This can be achieved using the **Fourier Transform**



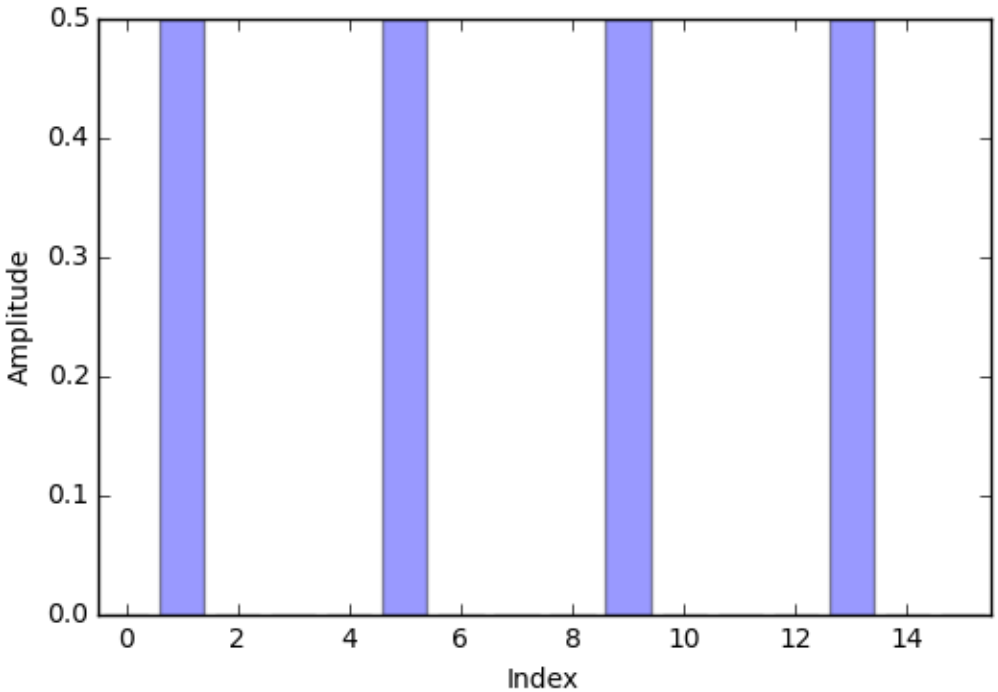


There is an efficient version of the Fourier Transform for quantum computers, which acts on states in *superposition*. This is known as the **Quantum Fourier Transform**.

The quantum Fourier transform

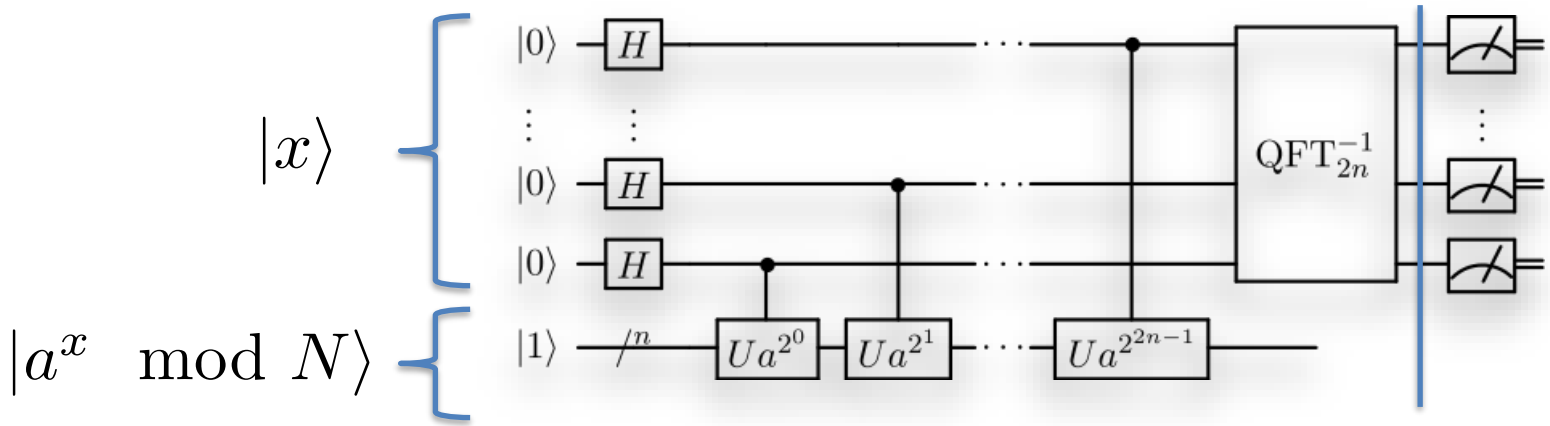
Imagine we measure the bottom register, and plot the amplitudes in the top register:

$$|\psi\rangle = (|1\rangle + |5\rangle + |9\rangle + |13\rangle) \otimes |2\rangle$$

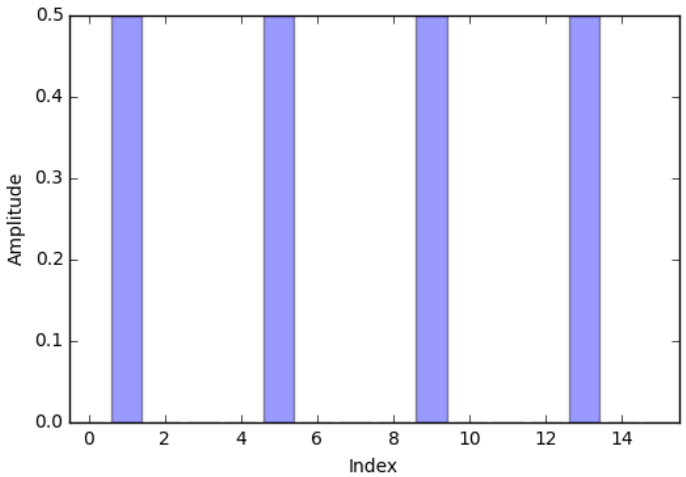


This function is periodic, with a period of r.

Taking the QFT



Inverse QFT for N=15, a=2

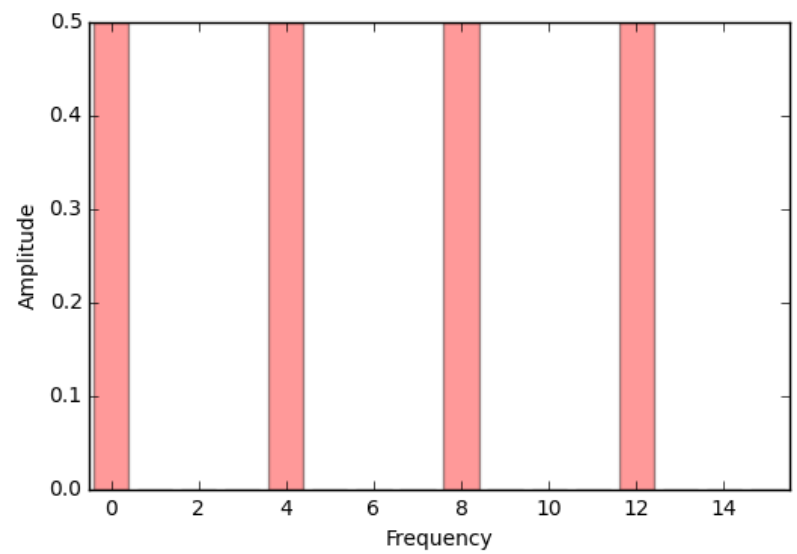


To find the period, we will take the Quantum Fourier Transform to reveal ***r*** (and so also, if *r* is even, factors of *N*).

The result of taking a Fourier transform is a "spectrum" peaked around (for integer, k):

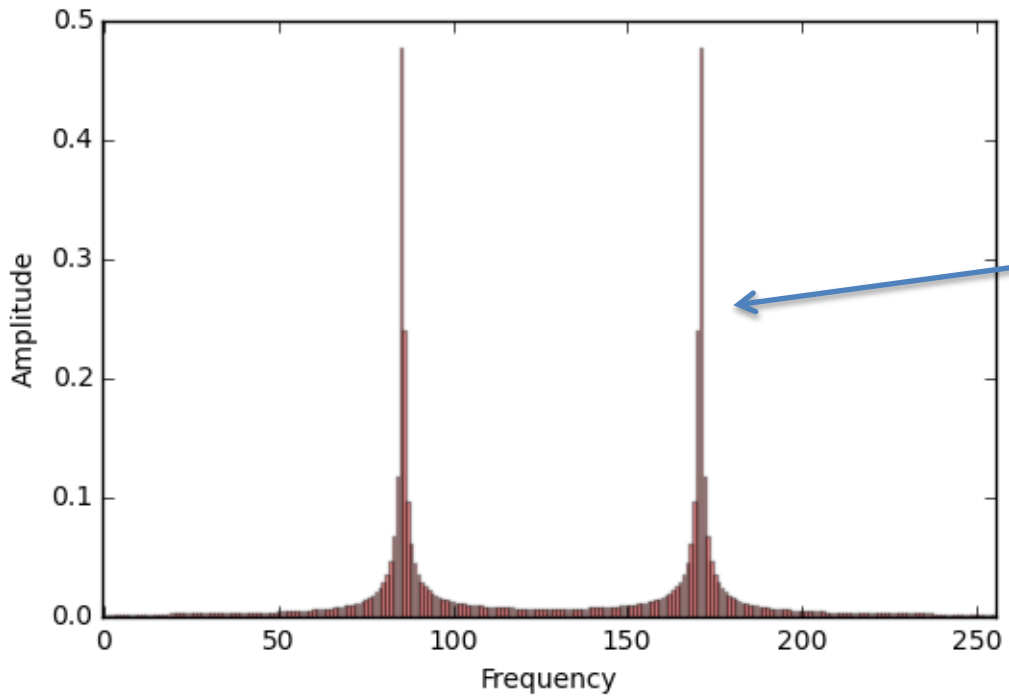
$$k\frac{2^n}{r}$$

n (2L) is number of qubits in the top register r is the period being determined



When r doesn't divide evenly

What happens when *r* doesn't divide evenly into the top register? Then we still get a very peaked distribution around the same values:



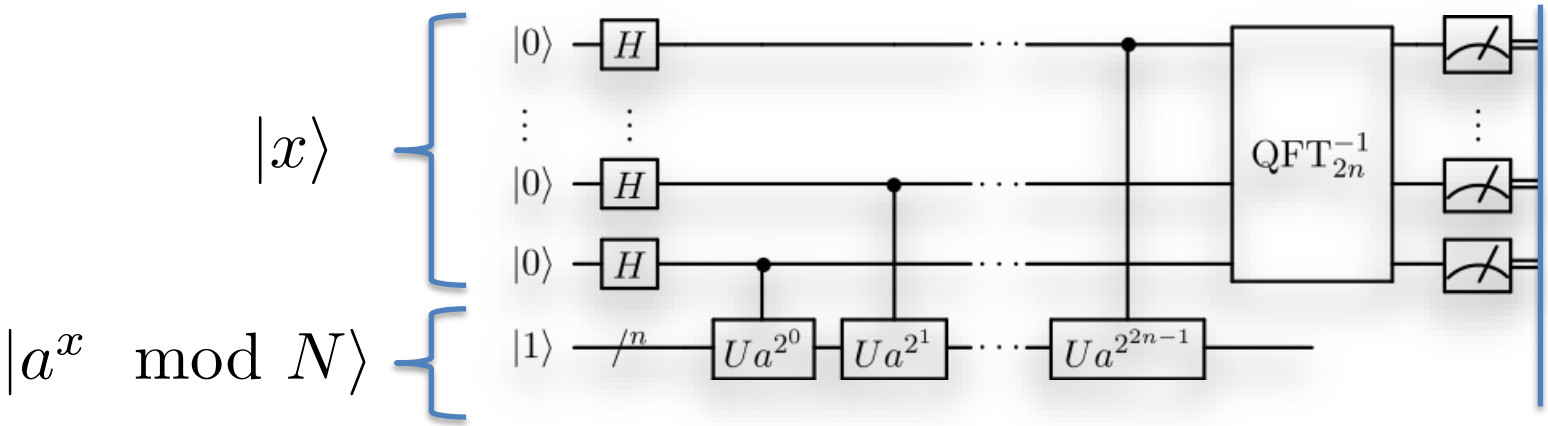
Peaked around:

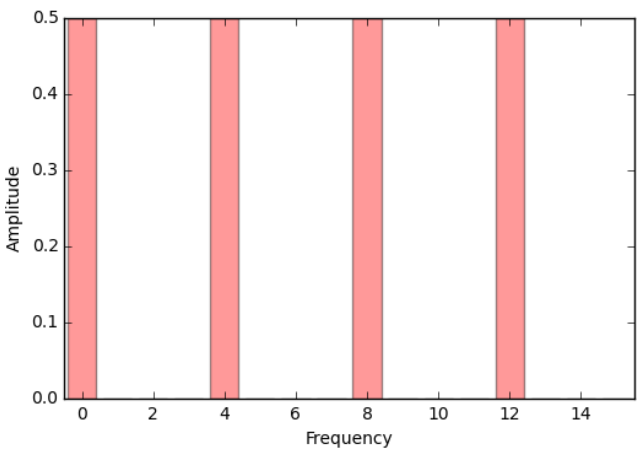
$$k\frac{2^n}{r}$$

Here is an example for $r=3$ and $2^n=256$.

13.3 Measurement and post-processing

Measurement





Measurement will randomly give one of these values, close to:

$$m = k\frac{2^n}{r}$$

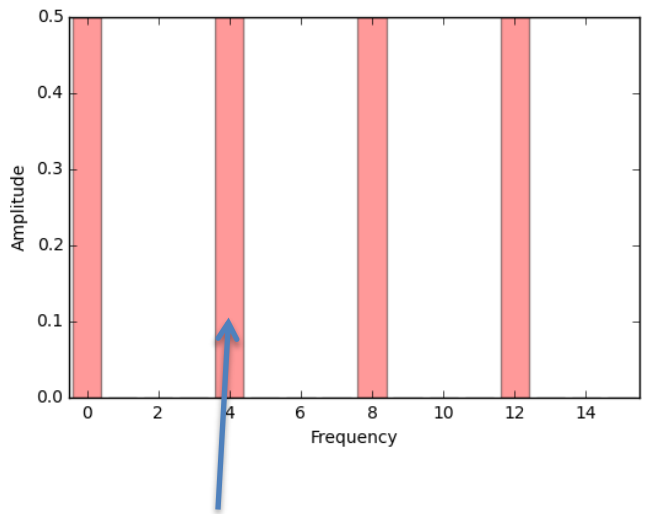
or

$$\frac{k}{r} = \frac{m}{2^n}$$

We need **a rational approximation** of $m/2^n$ to find r.

Example for a=2, N=15

In our example:



We might randomly measure m=4

$$\frac{k}{r} = \frac{m}{2^n}$$

and in this case:

$$\frac{m}{2^n} = \frac{4}{16} = \frac{1}{4}$$

Since this is equal to *k/r*,
We have correctly found
r=4

Note: This step might only reveal a factor of *r*, and so might have to be repeated…

Continued Fractions

The result of taking a Fourier transform is a spectrum peaked around (for integer, *k*):

$$k\frac{2^n}{r}$$

Unless r divides 2^n exactly, we will only get an approximation to $k2^n/r$ when measured.

Most of the time 2^n and r will be relatively prime. The problem then is find good approximations to the measured value $m/2^n = k/r$. The "correct" approximation yields the period, *r*, as the denominator.

A good method for making *rational approximations* is to use the **continued fractions** method.

$$a_0 + \cfrac{1}{a_1 + \cfrac{1}{a_2 + \cfrac{1}{\cdots \cfrac{1}{a_n}}}}$$

Continued Fraction: Example for Pi

As an example, let's try to make a rational approximation to *pi*. Our first approximation is

$$\pi \approx 3 \qquad (a_0 = 3)$$

The remaining decimal part is 0.14159265… = 1/7.0625… This gives a second approximation:

$$\pi \approx 3 + \frac{1}{7} \qquad (a_1 = 7)$$

The remaining decimal part 0.0625 = 1/15.9966… This gives a third approximation:

$$\pi \approx 3 + \frac{1}{7 + \frac{1}{15}} \qquad (a_2 = 15)$$

And so on. This method can be used to find good rational approximations to $v/2^n$ and find r.

Example: Finding 'q' using continued Fraction

As an example, let's assume we measure the value of m/2^n to be 7.28

7 + 28/100 = 7 + 1/(100/28) by rearrangement

= 7+ 1/(3+ 16/28)

= 7+1/(3+1/(28/16))

= 7+1/(3+1/(1+12/16)

= 7+1/(3+1/(1+1/(1+4/12)))

a_0=7, a_1=3, a_2=1, a_3=1, a_4=1, …

We could have stopped at a_3 to get 7+1/(3+ 1/(1+12/16))

by re-arrangement - reintroducing real fractions by multiplying through as follows

= 7+1/(3+1/(28/16))

= 7+1/(3+16/28)

= 7+1/(100/28)

= 7+28/100

= 728/100 = p/q, so p=728 and q = 100

So here try using denominator q from the third (rational) convergent of (real) numbers p_3 and q_3 as a potential candidate:

- Testing if a^q =1 mod N ?

If this is true, then q is likely the period r.

What if q is not even?

As an example, let's assume we measure the value of m/2^n to be 7.27

7 + 27/100 = 7 + 1/(100/27) by rearrangement

= 7+ 1/(3+ 19/27)

= 7+1/(3+1/(27/19))

= 7+1/(3+1/(1+8/19)

= 7+1/(3+1/(1+1/(2+3/8)))

a_0=7, a_1=3, a_2=1, a_3=2, a_4=2, …

We could have stopped at a_3 to get 7+1/(3+ 1/(1+1/2))

by re-arrangement - reintroducing real fractions by multiplying through as follows

= 7+1/(3+1/(3/2))

= 7+1/(3+2/3)

= 7+1/(11/3)

= 7+3/11

= 80/11 = p/q, so p=80 and q = 11

So here try using denominator q from the third (rational) convergent of (real) numbers p and q as a potential candidate for:

- Testing if a^q =1 mod N ?

But q is odd !

What if q is not even?

If the convergent has an odd **q**-value, then that **q**-value is not directly useful as the period **r** you're seeking, as you wish to use it in the following equations

$$(a^{r/2}+1)(a^{r/2}-1) = kN \qquad \frac{k}{r} = \frac{m}{2^n} = 7.27$$

- For this to work, you generally need ***r*** to be even for the algorithm to successfully factor

Sometimes, if the period ***r*** is odd, doubling it might yield the correct even period for the function.

- However, this doesn't always guarantee a successful factorization, and there are nuances involved.

Even if you get an odd q-value, it might still be close to the actual period *r*.

- For instance, if you find an odd *q* that's close to the true period, it might mean that your next convergent (or a subsequent one) could give an even q-value that is the actual period *r*.

Example: Factoring the number

$$a^{r/2} - 1 = 2^{4/2} - 1 = 3$$

$$a^{r/2} + 1 = 2^{4/2} + 1 = 5$$

Not really necessary here, but in general you'd have to evaluate:

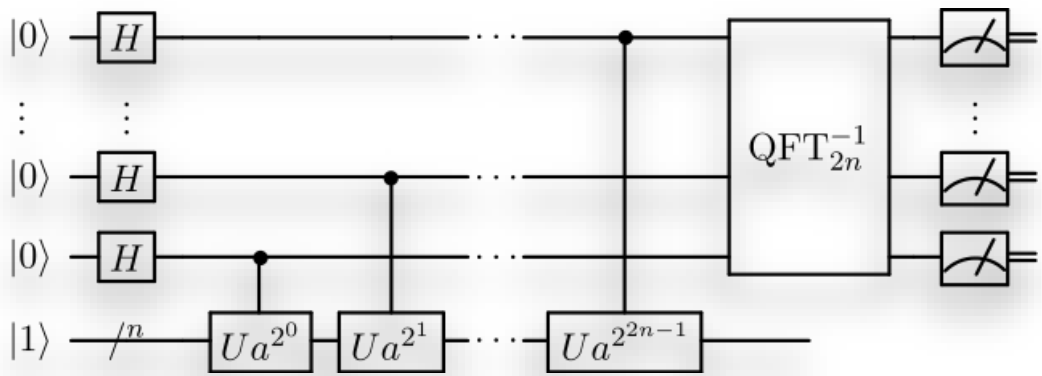
$$gcd(3, 15) = 3$$
$$gcd(5, 15) = 5$$

And so we've found two non-trivial factors of 15:

$$3 \times 5 = 15$$

13.4 Shor's algorithm summary

Shor's algorithm method

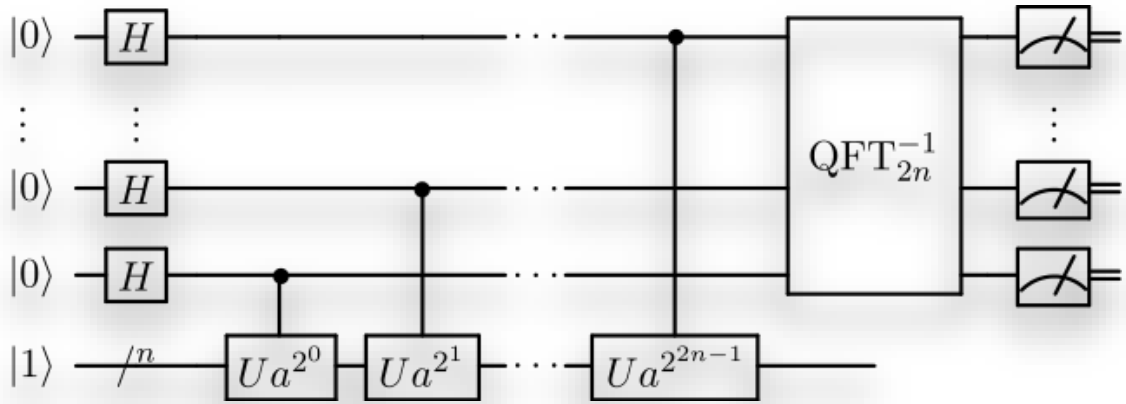


1. Randomly pick integer $0 < a < N$ (and check a coprime to N)
2. Use **quantum circuit** to calculate: $a^x \bmod N$, QFT x.
3. **Measure** to obtain and approximation to $m = k\, 2^n/r$
4. Use **continued fractions** of $m/2^{2n}$ to obtain even r
5. Use **Euclidean algorithm** to find *gcd* of N with $(a^{r/2}+1)$ and $(a^{r/2}-1)$
6. Repeat if necessary

Shor's algorithm summary

- Efficient quantum algorithms for **factoring** semiprime numbers
- Best known classical algorithm is number field sieve (exponential in bit-length).
- Underpins the RSA cryptosystem
- Hidden Subgroup Problems (eg. Discrete logarithm) similar.

Peter Shor



Shor, Proc 35th Ann Symp of Comp Sci, 26, (1995)

Subject outline

Lecture topics (by week)

1 – Introduction to quantum computing and maths basics
2 – Single qubit representations and logic operations
3 – Two qubit states and logic gates
4 – Multi-qubit states and quantum arithmetic
5 – Basic quantum algorithms
6 – Period finding, cryptography and quantum factoring
7 – Shor's algorithm, post-quantum crypto, quantum key distribution
8 – Quantum search algorithms
9 – Grover search applications, optimisation problems
10 – Solving optimisation problems on quantum computers
11 – Applications in quantum machine learning
12 – Real quantum computer devices

Assignment schedule:
#1: Hand out in Week 2
#2: Hand out in Week 8